
Loan Default — Probability of Default (PD) Model

This project builds an **Application-Stage Probability of Default (PD) model** for mortgage loans: it estimates the likelihood that a borrower will default, using only information that is available at the time of loan application. Variables generated later, during underwriting or loan pricing, are deliberately excluded so the model reflects a realistic lending decision process rather than one that benefits from hindsight.

Source data: [Kaggle Loan Default Dataset](#).

148,670 Loan Records	24.6% Default Rate	0.899 ROC-AUC (LightGBM)	74% / 77% Default Recall / Precision
--------------------------------	------------------------------	------------------------------------	--

Executive Summary

Lenders need to rank applicants by default risk **before** approval, using only what is known at application. This project trains four classifiers on 148,670 mortgage applications (24.6% defaulted) after removing post-underwriting pricing fields that would leak the outcome. **LightGBM** was selected as the final model, achieving an **AUC of 0.899**, a **Gini of 0.799**, and a **KS of 0.665**, and identifying 74% of actual defaulters at 77% precision. Observed default rates rise steadily across five PD risk bands, from 2.4% (Very Low) to 94.4% (Very High), showing the scores are suitable for approval, pricing, and portfolio-monitoring decisions.

Assumptions & Dataset

Assumptions

- The prediction is made before loan approval.
- Only application-stage information is used as model input.
- Post-underwriting and pricing variables are excluded.
- The model predicts probability of default, not probability of loan approval.

Dataset

- 148,670 rows × 34 columns.
- Target variable: Status (1 = Default, 0 = No Default).
- 112,031 performing loans vs. 36,639 defaulted loans (~24.6% default rate — moderately imbalanced).
- 13 numerical and 21 categorical columns spanning borrower demographics, loan terms, credit profile, collateral, and affordability.

Workflow

The analysis followed a ten-step notebook structure:

1. Import Libraries

2. Load & Inspect Data
3. Data Cleaning & Missing Value Analysis
4. Univariate Analysis
5. Bivariate Analysis (features vs. target)
6. Correlation Analysis
7. Outlier Detection
8. Key Insights Summary
9. Feature Engineering & Encoding
10. PD Modeling (train, evaluate, interpret)

Data Cleaning & Key Findings

Missing data hotspots

Column	Missing Count	Missing %
Upfront_charges	39,642	26.66%
Interest_rate_spread	36,639	24.64%
rate_of_interest	36,439	24.51%
dtir1	24,121	16.22%
LTV	15,098	10.16%
property_value	15,098	10.16%

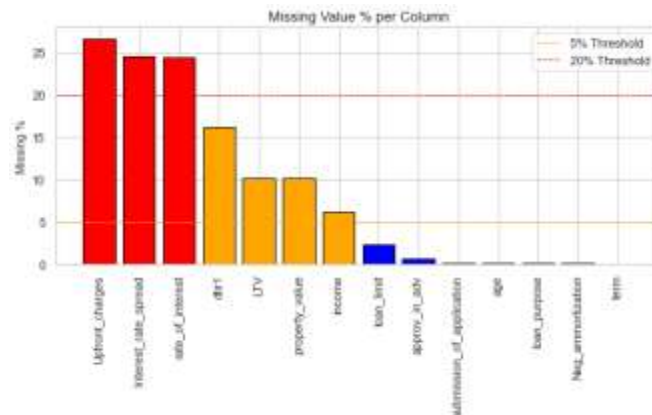


Figure 1. Missing value percentage by column.

Interpretation: rate_of_interest, Interest_rate_spread, and Upfront_charges are missing almost exclusively for defaulted loans, and are pricing fields set after underwriting — a live application-time model would never have them. Both facts led to their removal as a **structural, deterministic, pre-split decision**.

- credit_type was dropped separately because one category (EQUI) is an almost perfect proxy for default (~99.99%), which looks like a dataset artifact rather than a genuine credit signal.
- year was dropped as a constant (all rows = 2019) carrying no signal.

Correlation with default

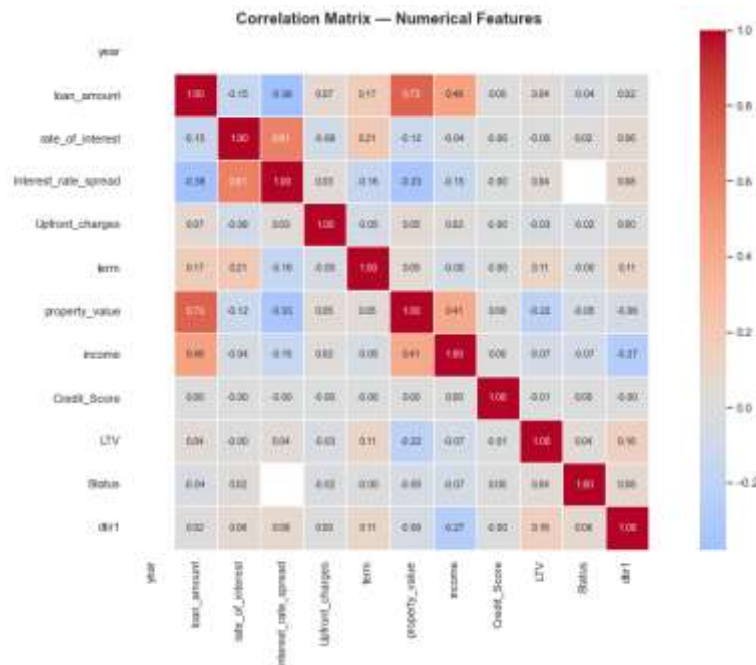


Figure 2. Correlation matrix of numerical features.

Top absolute correlations with Status: **dtir1 (+0.078)**, **income (-0.065)**, **property_value (-0.049)**, **LTV (+0.039)**, and **loan_amount (-0.037)**. Defaulters tend to show higher debt-to-income and loan-to-value ratios, and lower income and property value — all economically sensible directions.

Feature Engineering

- Performed deterministic, row-wise feature engineering by deriving property_value and LTV from their algebraic relationship with loan_amount, ensuring no information leakage across observations.
- Created NaN-aware risk indicators (high_dti for DTI > 40 and high_ltv_flag for LTV > 80), preserving missing values rather than assigning default risk flags.
- Engineered additional predictive features including loan_to_income and term_years to capture borrower affordability and loan duration.
- Converted continuous credit scores into five ordinal credit quality bands (Very Poor, Fair, Good, Very Good, and Exceptional) to improve interpretability and model performance.
- Applied explicit binary encoding to 13 binary categorical variables and one-hot encoding to nominal features (Gender, loan_type, loan_purpose, occupancy_type, total_units, Region, age, and credit_score_band).
- Standardized numerical features for Logistic Regression while leaving tree-based models unscaled, following model-specific preprocessing best practices.
- Executed all feature engineering, imputation, encoding, and scaling on the training data only, then applied the fitted transformations to the test set to eliminate preprocessing data leakage.

Modeling & Results

Four classifiers were trained on an 80/20 stratified train-test split, with class imbalance handled via `class_weight='balanced'` or `scale_pos_weight`:

Model	AUC	Gini	KS	Accuracy	Precision	Recall	F1	Brier
LightGBM (selected)	0.899	0.799	0.665	0.881	0.771	0.735	0.753	0.102
XGBoost	0.899	0.797	0.661	0.879	0.766	0.734	0.749	0.102
Random Forest	0.886	0.773	0.633	0.876	0.777	0.694	0.733	0.117
Logistic Regression	0.776	0.552	0.427	0.727	0.462	0.666	0.546	0.186

ROC-AUC was used as the primary selection metric because a PD model needs to rank borrowers by risk accurately, not just maximize accuracy. Accuracy alone can be misleading on an imbalanced credit dataset. **LightGBM was selected as the final model**, achieving the highest AUC, Gini, and KS statistic among all candidates.

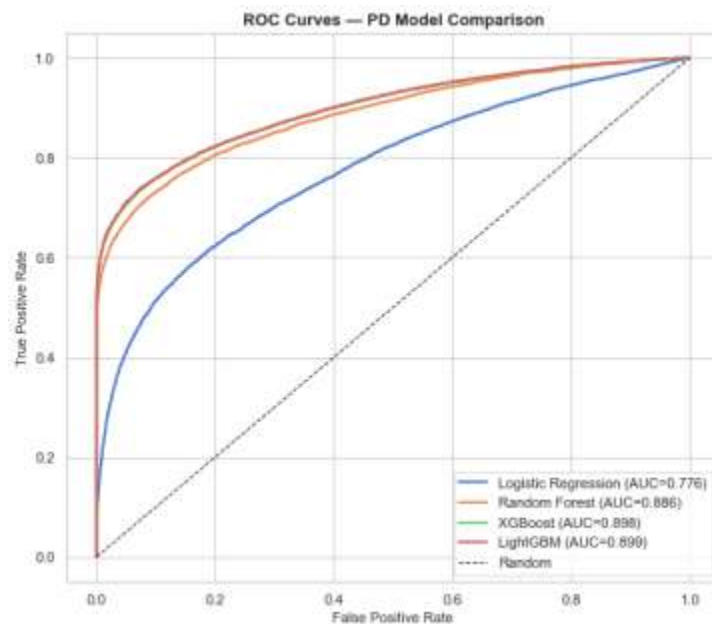


Figure 3. ROC curves for all four candidate models.

Final model performance (LightGBM, test set)

Class	Precision	Recall	F1-score	Support
No Default	0.91	0.93	0.92	22,406
Default	0.77	0.74	0.75	7,328
Accuracy	-	-	0.88	29,734
Macro avg	0.84	0.83	0.84	29,734
Weighted avg	0.88	0.88	0.88	29,734



Figure 4. Confusion matrix — LightGBM.

Recall is particularly important in PD modeling because failing to identify high-risk borrowers can be far more costly than a false alarm. The final model correctly identifies **74% of actual defaulters** (recall) while keeping **precision at 77%**, maintaining a reasonable balance between catching risky borrowers and minimizing false positives.

Feature importance

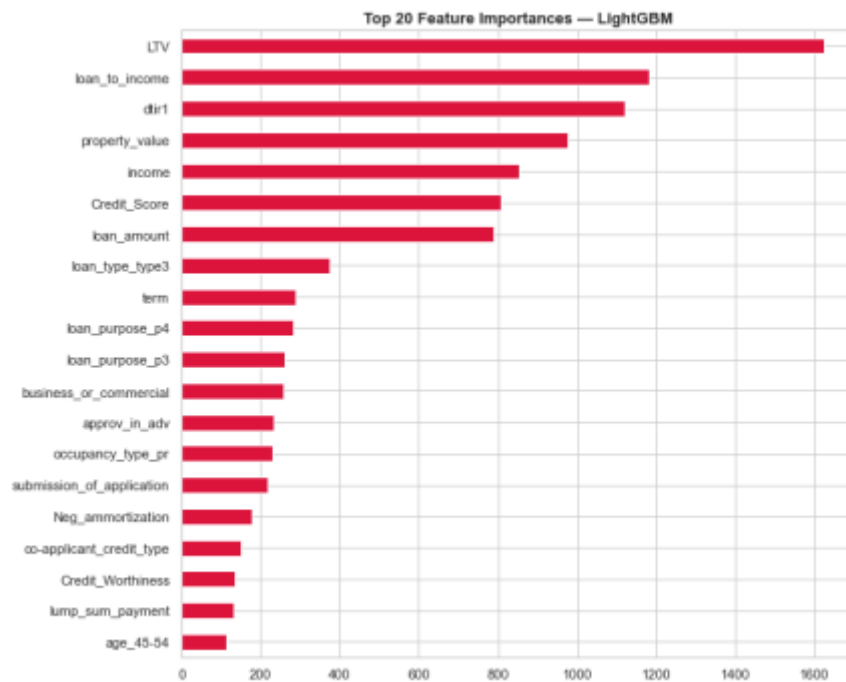


Figure 5. Top 20 feature importances — LightGBM.

LTV, loan_to_income, dtir1, property_value, income, and Credit_Score are the strongest drivers of predicted default risk — consistent with the correlation analysis and with standard credit-risk intuition. This supports model interpretability and regulatory documentation.

Calibration

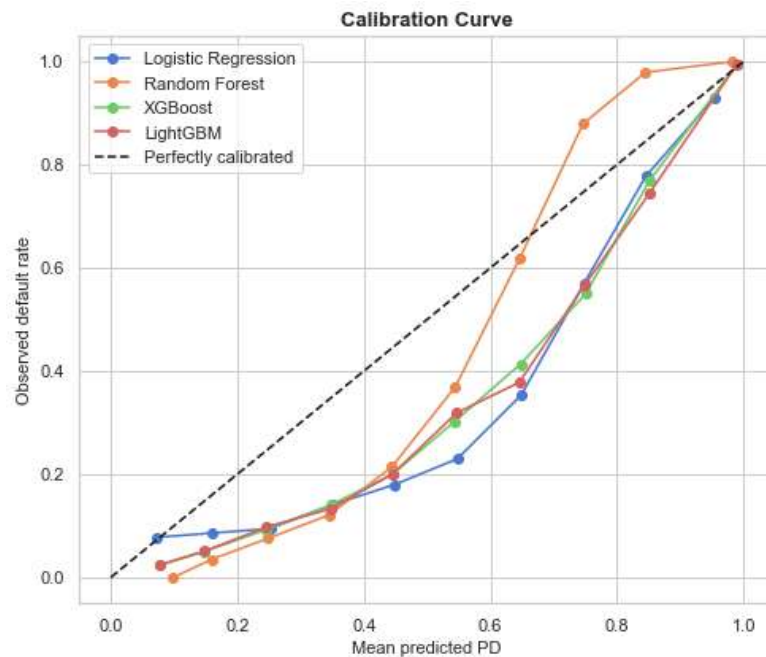


Figure 6. Calibration curve — predicted PD vs. observed default rate.

Calibration measures how closely predicted probabilities match observed default rates; curves near the diagonal indicate PD values that can be interpreted as realistic probabilities. Good calibration matters for **loan pricing, expected credit loss estimation, stress testing, and regulatory capital calculations.**

Score separation

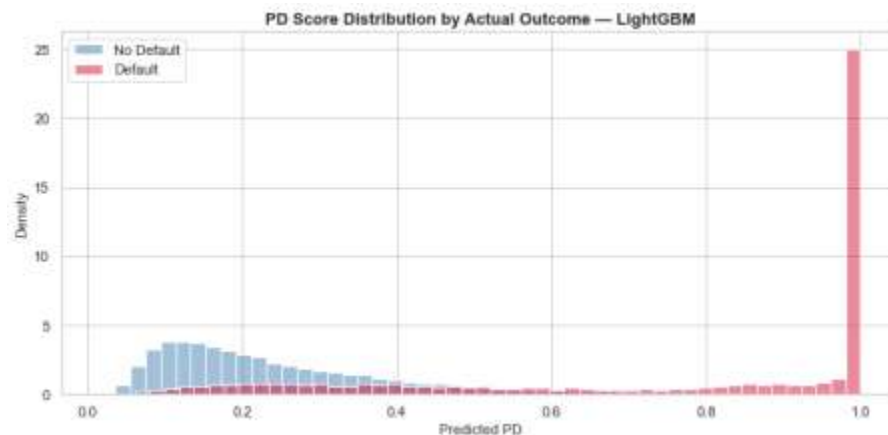


Figure 7. Predicted PD distribution by actual outcome — LightGBM.

Non-default borrowers concentrate at lower predicted PD; defaulters skew toward higher PD. The clear separation between the two distributions demonstrates the model's discriminatory power, while some overlap is expected since borrower characteristics are rarely perfectly separable.

Risk Bands

Borrowers were segmented into five PD bands to demonstrate practical usability:

Band	Count	Avg. Predicted PD	Actual Default Rate
Very Low	3,199	0.077	0.024
Low	11,457	0.169	0.061
Medium	8,095	0.351	0.144
High	2,070	0.603	0.362
Very High	4,913	0.961	0.944

The observed default rate rises monotonically from **2.4% (Very Low)** to **94.4% (Very High)**, confirming the PD estimates are meaningful and suitable for practical credit risk segmentation. Such risk bands can support:

- Loan approval decisions
- Risk-based pricing
- Portfolio monitoring
- Credit limit assignment
- Expected loss estimation

Tech Stack

- pandas, numpy, matplotlib, seaborn, scikit-learn (KNNImputer, OneHotEncoder, StandardScaler, LogisticRegression, RandomForestClassifier, train_test_split, metrics), xgboost, lightgbm.

Limitations & Notes

- The Security_Type variable contains a known typographical error ("Indriect" instead of "Indirect") in the original dataset, which was intentionally preserved to maintain consistency with the source data.
- All preprocessing steps — including feature engineering, imputation, encoding, and scaling — were fitted exclusively on the training dataset and subsequently applied to the test dataset, preventing data leakage.
- Model performance metrics were evaluated using the default classification threshold of 0.50, while the accompanying risk-band analysis illustrates model performance across a range of probability cutoffs for threshold sensitivity assessment.